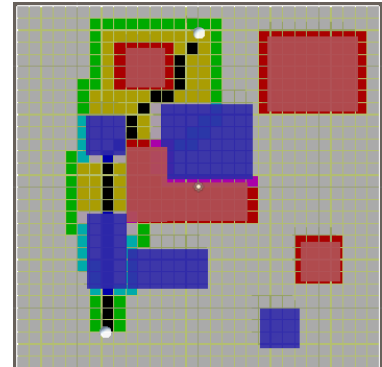
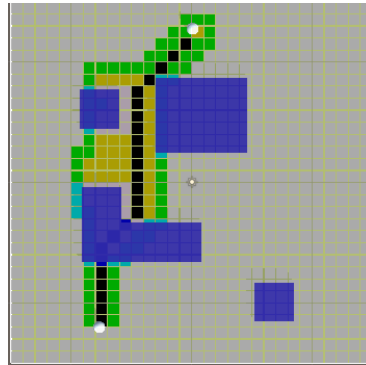
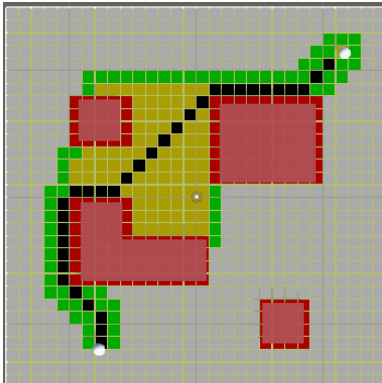


Patfinding A* Report:

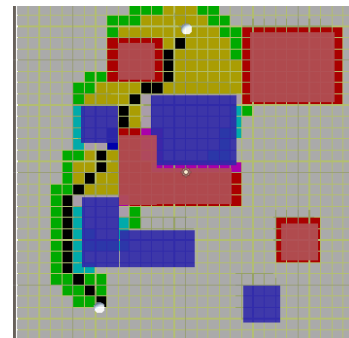
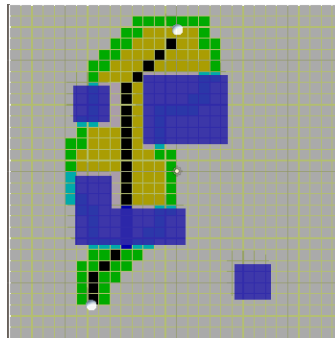
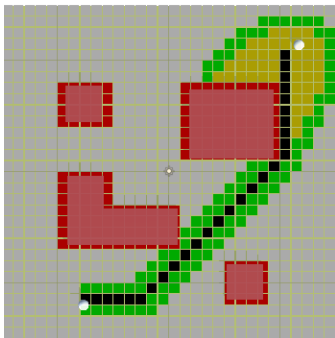
Implementing Pathfinding A*:

In a path-finding algorithm like A*, the heuristic estimates the remaining cost from the current node to the goal. If the heuristic does not properly account for the eight connectivity the algorithm can take incorrect or suboptimal paths and explore unnecessary nodes, making it slower.



If we don't check eight connectivity, it assumes movement is only possible in straight lines, causing it to explore many extra nodes. Making A* slower and inefficient as it considers unnecessary paths.

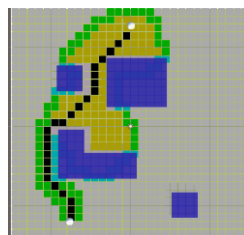
Checking eight connectivity:



```
float costMultiplier =  
    (Physics.CheckSphere(worldPoint, nodeRadius, costMultiplierMask)) ?  
    1.5f : 1.0f;
```

Is important to adjust the cost Multiplier, it may change the path avoiding the part with more cost:

If the costMultiplier = 2f :



Implimenting Smooth and Bresenham:

```
void SmoothPath(List<NodePathfinding> path)
{
    // Si el camino tiene menos de 2 nodos, no hay nada que suavizar
    if (path.Count < 2)
    {
        return;
    }

    List<NodePathfinding> smoothedPath = new List<NodePathfinding>();
    smoothedPath.Add(path[0]); // El primer nodo siempre está en el camino

    // Comienza con el primer nodo
    NodePathfinding lastValidNode = path[0];

    for (int i = 1; i < path.Count; i++)
    {
        // Verifica si el camino directo entre el último nodo válido
        // y el nodo actual es transitable y tiene un coste razonable
        if (BresenhamWalkable(lastValidNode.mGridX, lastValidNode.mGridY, path[i].mGridX, path[i].mGridY, lastValidNode.gCost))
        {
            // Si es transitable, no agregamos este nodo al camino y continuamos
            continue;
        }

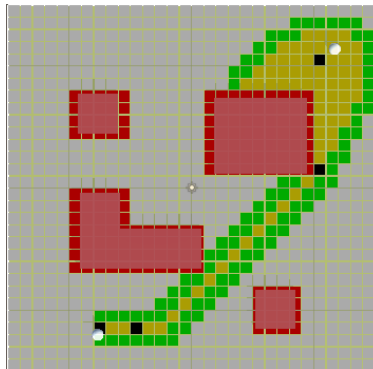
        // Si no es transitable, agregamos el último nodo válido al camino
        smoothedPath.Add(path[i - 1]);
        lastValidNode = path[i - 1];
    }

    // Siempre agregamos el último nodo del camino
    smoothedPath.Add(path[path.Count - 1]);

    // Reemplazamos el camino original con el suavizado
    path.Clear();
    path.AddRange(smoothedPath);
}

/*****
1 referencia
public bool BresenhamWalkable(int x, int y, int x2, int y2, float gCostStart)...
```

With the implementation of the smoothness and Bresenham, the path is composed of with the minimal nodes needed to define the smoother route.



Now, we have in account of the cost of the path to ensure that shortcuts do not pass through excessively costly terrain

```
}
int numerator = longest >> 1;
float totalGCost = 0;
float gCostThreshold = gCostStart * 0.1f;

for (int i = 0; i <= longest; i++)
{
    NodePathfinding node = Grid.GetNode(x, y);
    totalGCost += node.mCostMultiplier;

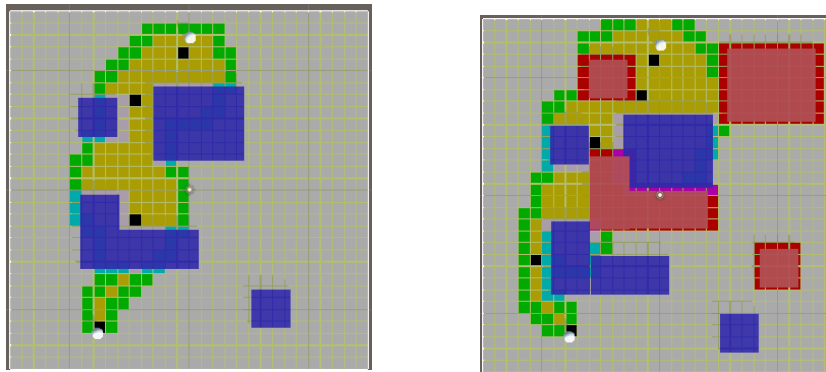
    if (!node.mWalkable || (node.mCostMultiplier > 1f && totalGCost > gCostThreshold) || !IsFourConnected(x, y))
    {
        return false;
    }
}
```

The cost of the traversed nodes is accumulated: $\text{totalGCost} += \text{node.mCostMultiplier}$;

Each node has a cost multiplier (mCostMultiplier), which may be greater than 1 if it represents difficult terrain.

Then, it checks if the current node has a cost greater than 1 (meaning it is not normal terrain). And if the total accumulated cost (totalGCost) exceeds the threshold (gCostThreshold), the path is considered too expensive. So, the route is rejected by returning false.

Resulting:



By increasing the gCostThreshold , there are fewer nodes being discarded because there are explored more nodes before the accumulated cost disqualifies them.

This makes key nodes in the smoothing process less apparent, potentially making it harder to identify the most critical ones for path optimization.

